

SabiPass AI — Master Context Document

Last updated: 2026-07-27 **Purpose:** Paste or upload this entire file at the start of any new chat (any account, any Claude session) to restore full project context, architectural decisions, and working method. This file is the actual persistence mechanism — Claude's built-in memory is account/project-scoped and will NOT transfer across accounts, so this document is what makes the project portable.

0. How to use this document

Upload this file at the start of a new session and say something like: *"This is the current state of SabiPass AI. Read it fully, then let's continue from [whichever open item]."*

Everything below is either a **locked decision** (treat as settled, don't silently re-propose alternatives) or an explicitly **open item** (fair game to discuss).

1. Claude's role & working method (read this first)

- Act as a **critical technical co-architect**, not a yes-man. Push back on ideas, name risks, flag scope creep, flag exploit surfaces. Never blindly agree.
 - **One decision thread at a time.** Close it with an explicit rationale before moving to the next. Don't try to solve five things in one reply.
 - **Decision log format:** `Decision: [what] – Why: [rationale] – Status: active` (or `superseded by X`).
 - If something is undecided, say so explicitly — mark it "Deferred — blocked on X" rather than leaving it ambiguous or assuming an answer.
 - The user is a **solo, non-CS-background developer** (civil engineering student, University of Ibadan) who understands system design and API/RAG concepts well but does not write code from scratch — code gets AI-generated and reviewed, not hand-authored. Treat this as a strength (architectural judgment) not a gap to apologize around, but keep implementation guidance concrete enough to hand to an AI coding agent.
 - Budget reality: **self-funded student**, zero external investment, zero industry connections. Every infra/tooling recommendation must default to the cheapest defensible option, not the "best practice at scale" option, unless explicitly asked to plan for scale.
-

2. Project overview

SabiPass AI — an offline-first AI tutor for Nigerian secondary school students (SS1-SS3) and JAMB/UTME aspirants, aligned to the NERDC curriculum and WAEC/JAMB/NECO past questions.

Core differentiators vs. generic AI (ChatGPT/Meta AI/WhatsApp AI) and vs. incumbents (MySchool CBT, etc.):

- **Socratic, never spoon-feeds answers** — guides students to the answer instead of solving it for them. This is a parent-facing trust argument as much as a pedagogy one (parents don't want their kid using AI to cheat).
 - **Two distinct engine modes per subject type**, not one generic chat:
 - **Socratic Solver** — analytical subjects (Math, Physics, Chemistry, Accounting): strict, step-by-step, hint-based, checks reasoning before advancing.
 - **Reading Partner** — narrative/descriptive subjects (History, Government, Literature, Biology theory): conversational "know-it-all senior classmate" that quizzes, engages, and uses active recall instead of passive reading.
 - **Curriculum-guided, not open-ended** — mastery tracked via micro-skills mapped to NERDC/WAEC/JAMB, not a blank chat box.
 - **pyBKT (Bayesian Knowledge Tracing)** as the "brain" tracking real mastery probability (accounting for guess/slip), separate from the LLM as the "voice." The LLM never self-assesses whether a student has learned something — pyBKT does, from behavioral data only (see guardrail in §5).
 - **Offline-first, cloud-augmented hybrid**: CBT practice and flashcards work fully offline; AI remediation, pyBKT updates, and deep personalization happen when the device reconnects (queued sync, not blocking).
-

3. Confirmed architecture decisions

3.1 Backend split — LOCKED, final

- **Node.js** = public-facing backend/gateway (auth, business logic, routing, wallet/payment logic when built).
- **FastAPI (Python)** = stateless AI microservice, communicates with Node.js via **gRPC**.
- The FastAPI microservice is **never publicly exposed** — no public port mapping. Achieved via Docker internal networking (both services in the same Docker Compose internal network; only Node.js exposes a public port). This gives full private-network security on a single cheap VPS — no need for two separate physical servers.
- Rationale: security isolation without doubling hosting cost; stateless AI service can later be moved to its own server without any application code changes if it ever needs to scale independently.

3.2 Hosting & infra — locked direction, some specifics still tunable

- **Rejected:** Neon (free tier compute-hour limits make it unviable for 24/7 production; paid tier too expensive for a bootstrapped budget), Supabase (flat \$25/mo floor is a fixed liability before revenue exists), Chroma as a second database (operational

complexity of two DBs — distributed transaction risk, no ACID across them, doubles backup/maintenance burden).

- **Chosen direction:** Self-hosted stack on a cheap VPS (Contabo or Hetzner, ~€5-6/mo), Docker Compose (explicitly **not** Kubernetes for MVP — Kubernetes is overkill for a solo dev and wastes RAM/time on orchestration complexity that isn't needed until far higher scale).
- **Nigerian-specific operational risks flagged (real, not hypothetical):**
 - Hetzner/Contabo are strict on KYC — use real Nigerian ID (NIN/passport), no VPN during signup, use a dedicated virtual USD card (Chipper Cash/Grey), pay quarterly where possible to avoid renewal-failure suspensions.
 - "Noisy neighbor" risk on budget VPS providers (oversold CPU) — monitor CPU steal time; have Hetzner as a backup option if Contabo underperforms.
- Database data must be **Docker-volume-mapped to host storage**, never left inside the container (container restarts must not wipe data).
- Automated Postgres backups via cron → Cloudflare R2 (free egress). Frequency: every 4 hours, not just nightly, prefixed with `(nice)`/`(ionice)` so backups don't compete with live student traffic for CPU.

3.3 Database — partially locked

- **Locked:** Single self-hosted **PostgreSQL** instance handles relational data (users, auth, wallet, pyBKT mastery state, sync queue). No second relational DB.
- **Open (see §4.1):** Whether pgvector is used for embeddings, whether one DB or a split (Postgres + dedicated vector DB like Qdrant) is used, and the actual RAG retrieval pattern. Past-question matching for CBT (cross-topic tagging) is currently designed as a **relational multi-tag filter**, not vector search — this part is settled regardless of the pgvector question (see §3.6).
- Local on-device: **SQLite** for offline CBT question banks and student local state (AsyncStorage/SQLite depending on layer), synced to R2 as a master file with hot-swap (no app restart needed to pull new question banks — an internal admin endpoint swaps the file in memory).

3.4 Static asset storage — LOCKED

- **Cloudflare R2** for textbook content (converted to Markdown/HTML) and past-question banks. Zero egress fees, 10GB free tier. Chosen specifically to avoid Neon/Postgres storage costs ballooning from housing large static text blobs in relational rows.
- Textbook PDF → Markdown → HTML pipeline (not direct PDF→HTML) to preserve structure, tables, and LaTeX math cleanly. Never scrape/store textbook text verbatim long-term — use it to inform NERDC-aligned proprietary explanations written by the

pipeline/LLM (copyright risk mitigation). **WAEC/JAMB past questions are public domain and safe to use directly.**

3.5 Offline/online sync model — LOCKED

- CBT practice, flashcards, and basic scoring work **100% offline** on-device.
- While offline: rich telemetry queued locally (time per question, option switches, confidence rating, not just right/wrong).
- On reconnect: background sync pushes the queued batch to FastAPI, which runs it through **pyBKT** to update real mastery probabilities, queries the curriculum store for targeted remediation, and pushes personalized content (Sabi Cards, Socratic hints, garden growth updates) back down.
- This is the core "futuristic" bet — heavy AI/Bayesian computation stays server-side (full Python ecosystem), mobile client stays lightweight and battery-friendly.

3.6 AI cost & routing strategy — locked principles, routing specifics open

- **DeepSeek** as primary paid model (cheap, large context, strong prompt-caching discount — ~98% off cached tokens). Model tier (Flash vs Pro) split by subject/task complexity is a real idea but **not finalized** — see §4.1.
- **Groq (Llama, free/cheap tier)** used as:
 1. An **input guardrail/bouncer** — cheap model screens for gibberish, abuse, or off-topic input *before* it ever reaches the paid model, so trolling/spam costs ~\$0. Must be tolerant of genuine typos/slang/shorthand — only block actual gibberish or malicious input, not imperfect English.
 2. A **failover**, not a co-equal engine — if DeepSeek has an outage/latency spike, requests degrade gracefully to Groq rather than the app breaking.
- **Vision decoupled from reasoning**: an image (homework photo, past-question scan) is first run through a cheap vision/OCR pass that transcribes it to clean text/LaTeX only (does not solve or explain) — then that clean text is what gets passed to the reasoning model. This keeps image tokens cheap and lets the text portion benefit from prompt caching, which raw image tokens can't.
- Past-question cross-topic tagging (a single WAEC question can span multiple syllabus topics) is handled via a **relational multi-tag matrix + progression filter** (question tagged with an array of topic IDs; only surfaced once the student has covered all tagged topics), not via vector similarity — this was explicitly chosen to avoid needing pgvector just for this feature.

3.7 Legal/compliance flags (not yet resolved, but must not be forgotten)

- Users under 18 are legally minors under Nigeria's NDPA. A guardian-contact mechanism is needed (framed positively — "so we can share your wins," not as a compliance checkbox) but **exact timing/copy is undecided** (see §4.6).

- `device_id` used for single-active-device enforcement is currently trusted at face value from the client — **spoofing risk is real and explicitly deferred** to a future Auth & Session Management design pass, not solved by the onboarding flow.
-

4. Open / deferred — explicitly unresolved

These are real gaps, not just "haven't gotten to it yet" — do not assume answers for these in future sessions.

4.1 RAG / vector / long-term memory architecture — highest priority open item

Blocked on a dedicated session. Needs to resolve, together, not piecemeal:

- pgvector vs. no vector DB at all vs. a dedicated vector DB (Qdrant free tier was raised as an option) — one database or two.
- The actual RAG retrieval pattern for textbook content (how chunks are selected/ranked, not just where they're stored).
- Long-term student memory beyond pyBKT's mastery matrix — is a knowledge-graph layer (LangGraph was explored, then explicitly deprioritized as "framework bloat" for a raw-API, cost-conscious backend) actually needed, or does a simpler structured state table suffice? Current lean: raw code + a compact JSON state object per student, not a graph framework — but this was never formally closed as a decision, just where the conversation drifted.
- How to maximize DeepSeek prompt-cache economics given whatever retrieval pattern is chosen (static system-prompt prefix stability matters a lot here).
- Model routing specifics (which subjects/tasks get Flash vs Pro) — raised multiple times, never finalized.

Do not let this stay vague — it drives the cost model, which blocks monetization (§4.2).

4.2 Monetization — blocked on §4.1

Cannot be decided until real cost-per-active-user is known (hosting is roughly fixed/cheap; token cost is the variable that depends on §4.1's retrieval pattern). Constraints given by the user, to hold onto for when this gets designed:

- Must not be so expensive students defect to free tools (Meta AI/ChatGPT) — SabiPass's actual moat is Socratic-not-spoofed + curriculum alignment + mastery tracking, not raw answer generation, so pricing has to reflect that value, not compete on raw AI access.
- Must not bankrupt the student-founder's own pocket while bootstrapping.
- Must not make students feel cheated/nickel-and-dimed by upsells.
- Should be "crazy cheap but still profitable" — Temu-style economics were explicitly cited as the psychological model to study (not the referral/gamification mechanics —

see §4.3 — specifically the *pricing perception* trick of making something feel like you're "getting away with it" while the unit economics still work).

- Models explored but **none chosen**: flat monthly subscription (₦500-1000), "Season Pass" tied to exam date, "Academic Year Pass," Pay-As-You-Go micro-bundles (₦500/50 scans, ₦1000/120 scans) — the micro-bundle model got the most favorable math (~64-70% margin at ~1.35-3 NGN cost per AI scan on Flash-tier pricing) but this was exploratory, not settled.
- Referral/sharing mechanics for user growth — explicitly "method not discussed yet."

4.3 Growth loop / viral mechanics — status ambiguous, needs explicit resolution

Two different philosophies were explored and never formally reconciled:

- **Early direction (Gemini session)**: Temu-style dopamine onboarding — spin-a-wheel "you've already won" framing, referral-gated unlock, viral loop baked into onboarding itself.
- **Later direction (this session, locked for onboarding entry logic — see §5)**: calm, non-gamified, anxiety-reducing onboarding; the "locked dashboard preview" is the hook, not a wheel-spin trick; no persistent/gamified guest mode.
- **Current read**: the calm/locked-preview approach is what's actually been designed and decided for onboarding (§5). The Temu-style viral/referral growth mechanic hasn't been killed, but it also doesn't currently have a home in the onboarding flow as designed. If viral/referral growth is still wanted, it likely needs to live *elsewhere* (e.g., post-mastery moments, streak-protection framing) rather than at the account-creation moment, which is now designed around trust/calm, not urgency/gamification.
This needs an explicit user decision, not an assumption, next time it comes up.

4.4 CBT engine — UX substantially designed, not reconciled with newer decisions

Designed: offline-first, Standard vs. custom "Syllabus Sculptor" mode (subject/topic/time-boxed practice, hard 4-hour time cap), anti-cheat relative-tick timer (not wall-clock, to prevent clock manipulation), network-drop handling (non-invasive banner, not a blocking error), intentional hold-to-confirm submission gate, post-exam AI remediation bridge, opt-out toggle for "update my AI vs. keep this attempt private." **Not yet reconciled** with today's onboarding/auth decisions (e.g., does a locked/no-account state get any CBT access at all, given §5's "everything locked until signup" rule?) — worth an explicit pass.

4.5 Homework Helper (premium vision feature) — conceptually designed, not finalized

Vision-based, Socratic (crops to one step at a time, never gives the final answer), personalizes using the student's learning anchor and current mastery level, logs recurring mistake patterns to a background "Mistake Ledger" that quietly feeds back into other parts of the app. Gating/pricing tied to §4.2.

4.6 Parent/guardian loop — conceptually designed, mechanics/timing open

Weekly WhatsApp-style summary framed as celebration, not surveillance ("scored 85%, 4-day streak" — not raw pass/fail judgment). Ties directly into the NDPA minor-consent question in §3.7 — **exact onboarding moment where a guardian contact is requested has not been decided** (earlier drafts placed it right after DOB capture, which was explicitly rejected as violating reciprocity/too-early-an-ask; a later placement post-dashboard-unlock was proposed but never confirmed as final).

4.7 Dashboard — concept only ("Sabi Garden" growth metaphor, locked-preview-until-signup), no detailed layout/mechanics designed yet.

4.8 Sabi Cards (flashcards) — spun off as an independent feature (deliberately decoupled from CBT so "testing" and "memorizing" stay separate mental modes), auto-populated from CBT mistakes via a background worker (fires after exam submission, not synchronously). Deck browsing/spaced-repetition mechanics not fully specified.

5. Onboarding flow — the most fully specified module (locked as of 2026-07-25)

5.1 Interaction style — explicitly reconciled this session

Two versions existed in prior history: an early **conversational** design (SP as a character, typewriter animation, dialogue-driven data capture, warm/witty tone, reciprocity-timed auth) and a later **linear numbered-steps "Production Blueprint"** the user wrote themselves. **Resolution: the Blueprint's step content/sequence is canonical for what gets captured and in what order; the conversational SP style is canonical for how it's delivered.** I.e., build the 9 steps below, but each step is a dialogue turn with SP, not a sterile form screen.

5.2 Entry logic — locked in detail this session

App Launch

→ Check local onboarding-progress flag (silent, on-device, no question asked if answerable locally)

IF flag absent:

- Ask: "New here, or already have an account?"
 - New → begin interview (Step 1 onward, §5.3)
 - Existing → email login (strictly email, no SMS/phone OTP)
 - Not found → rate-limited retry → after limit reached, offer "start as new" with the email they typed pre-filled (don't make them re-type it) → begin interview
 - Found → check device_id
 - differs from currently-bound device → revoke old session, bind new device
 - same or none bound → bind device
 - auto-login → Dashboard
- (Smart Merge: server-side mastery/garden/history data always wins and is never overwritten; if this login flow originated from a completed *new-user* interview that hit an existing email at Step 9 instead, the freshly-entered class/subjects DO get applied on top)

IF flag present, onboarding was completed:

- straight to Dashboard

IF flag present, onboarding incomplete:

- gap since last activity < 7 days → silently resume at the exact step they stopped on (no re-ask)
- gap ≥ 7 days → don't discard progress, but re-confirm class/subjects with a lightweight prompt before continuing (not a full restart) — the 7-day threshold is a starting guess, explicitly flagged as tunable once real usage data exists, not a permanent constant

Progress is persisted **locally, after every single answered question** (not just at checkpoints) — cheap (local write, no network call), and means nothing is ever lost to an app close/crash mid-interview.

5.3 The 9-step interview content (delivered conversationally, per §5.1)

1. **Welcome + trust framing** — calm, non-intimidating tone ("SabiPass helps you learn fast without stress," "no copying, we guide you to solve").

2. **Fast account creation OR the new/existing fork from §5.2** happens around here (exact conversational placement of the fork vs. this being step 2 proper needs a quick confirm next session, low-stakes).
3. **Class level** (SS1/SS2/SS3) — non-negotiable, primary curriculum guardrail. If JAMBite/UTME-focused instead of a current class, this branches the rest of the interview's framing (dream course → auto-derived 4-subject UTME combo → prior score capture, framed neutrally: "no bad number here, it's just where we start counting from").
4. **Subject selection** — multi-select, becomes "active subjects." **Enter Sabi Garden cannot be pressed with zero subjects selected** — blocked upstream, not a dead-end error later.
5. **Learning anchors** — 1-3 relatable real-world domains (football, cooking, tech/gaming, local markets) used for analogies in later tutoring. Framed as an invitation ("what part of your world makes it click?"), not a ranked/graded choice. Skipping defaults to "everyday life" analogies, not a hard block.
6. **Exam focus** (WAEC/NECO/JAMB, multi-select, optional) — skipping defaults to *all three enabled*, not a dead end.
7. **Confidence check** — one combined screen listing all selected subjects with a Low/Medium/High selector each (not one screen per subject).
8. **Permissions & preferences** — notifications (**true OS-level interrupt, drawn as a system-interrupt node**, guests/pre-signup users skip this since there's no push token to register yet), offline-download toggle (**this is a normal in-app toggle, NOT an OS permission prompt** — saving to the app's own sandboxed storage doesn't trigger an OS dialog on iOS or Android), data-saver toggle.
9. **Final confirmation** ("You're in SS2, studying Physics & Chemistry — ready?") → **Enter Sabi Garden CTA**.

5.4 "Guest mode" — resolved, does NOT exist as a persistent feature

There is no indefinite anonymous-usage tier. What exists instead: the **interview itself (steps 1-8) is free and un-gated** — this is what builds investment (reciprocity/IKEA-effect reasoning carried over from the earlier conversational design). The **Dashboard is a locked preview** — visible in outline, but tapping into anything functional bounces back to the signup wall. Account creation is mandatory before the app does anything actually usable. This removes the need for any anonymous-data merge system, sync-on-convert logic, or risk of silent data loss — there's nothing to lose because nothing persists until there's an account.

5.5 Slot-filling cost control — locked

Any AI-assisted extraction during onboarding (e.g., parsing a spoken/typed "what's giving you trouble" answer into structured weakness tags) runs through a **two-layer filter before ever touching the paid model**:

1. Local on-device keyword/dictionary match against known syllabus terms — free, no network call, catches the majority of cases.
2. Only if that's inconclusive, fall through to the cheap/free-tier model (Groq/Llama), never straight to DeepSeek. This means most onboarding installs cost effectively \$0 in AI spend, which matters directly for the "crazy cheap" economics goal before a student has converted to paying.

5.6 Correction/editing UX — locked

No dedicated "back" navigation through the interview. Every already-given answer renders as a **tappable, inline-editable chat bubble** — tap it, retype, conversation continues forward from where it was. No undo stack, no multi-screen back button.

5.7 System-wide guardrail (not onboarding-specific, but surfaced here) — important, keep visible

Self-reported data (weaknesses, confidence levels, anything typed/spoken by the student) may only ever influence *personalization* — never gate a reward, discount, bonus currency, or unlock. Anything reward-adjacent must be driven by *behavioral* data (actual quiz performance, pyBKT-derived mastery) instead, because self-report is trivially gameable the moment it pays out — students will learn and spam whatever input triggers the best reward. This is the same failure class as the earlier "type gibberish 3 times to skip" escape hatch, which is safe *only* because skipping leads to a neutral default, not a reward.

6. Tooling & meta-workflow notes

- Architecture discussions with Claude/Kimi happen in **standard web chat**, not Claude Code CLI/terminal agents — this matters because most of the "12 popular Claude skills" circulating online (GitHub MCP, LSPs, Playwright, etc.) require terminal/filesystem access and are **not usable** in this context. What *does* transfer as pure behavioral discipline (no install needed, just stated as working instructions): **Karpathy-style rigor** (think before designing, simplicity first, goal-driven, state assumptions explicitly when ambiguous) and **Superpowers-style phase isolation** (one atomic decision at a time, don't let the model race ahead to solve the whole system in one pass).
 - Because a web chat has no persistent filesystem, the practical substitute for "skills" here is: **checkpoint discipline** — when a sub-piece of architecture is agreed, get a small, isolated markdown snippet for just that piece (not a full re-write of the whole spec), copy it into the running local document, and start fresh chats once a thread gets long rather than letting context degrade.
 - This document *is* that running local file. Keep it updated the same way — append/amend sections as decisions close, rather than starting over each time.
-

7. Condensed history — key ideas from the original Gemini brainstorming session

(Full raw transcript not reproduced here — this is a cleaned, thematic summary of everything explored before the Claude session began. Treat items here as either already absorbed into §3–§5 above, or as raw exploratory material for the open items in §4 — not as additional locked decisions.)

- **RAG strategy landscape reviewed early on** (re-ranking, agentic RAG, knowledge graphs, contextual retrieval, query expansion, multi-query, context-aware/late chunking, hierarchical RAG, self-reflective RAG, fine-tuned embeddings) — informs §4.1 but no specific pattern was chosen from this list.
- **pyBKT explained and adopted** as the "cold mathematical auditor" layer, explicitly separate from the LLM's conversational role ("separation of church and state" framing) — this became §2's core pedagogical model.
- **Extensive infra cost research:** Neon vs. Supabase vs. Firebase vs. self-hosted Postgres, Contabo vs. Hetzner, Kubernetes vs. Docker Compose, SQLite+Litestream as an ultra-lean alternative — all folded into §3.2–3.3's locked direction.
- **Textbook ingestion pipeline:** PDF → Docling/Marker → Markdown → chunked HTML by curriculum node, explicitly rejecting manual PDF-to-HTML conversion as unworkable at 2000+ page textbook scale — folded into §3.4.
- **Pricing/monetization brainstorm** (sachet/micro-credit model inspired by Nigerian data-bundle purchasing habits, "Energy Bar" abstracted currency instead of raw Naira-per-token to avoid parent sticker-shock, Season Pass vs. subscription vs. one-time device-lifetime PIN) — all still exploratory, feeding §4.2, none chosen.
- **Temu-inspired growth psychology** (goal-gradient artificial head-start, loss-aversion streak framing, reciprocity-before-paywall, IKEA-effect self-built profiles, contrast-effect price anchoring) — the *psychological principles* were broadly adopted (visible throughout §5's onboarding design), but the specific *viral/gamified mechanic* (spin-wheel, referral-gated unlock) was NOT carried into the final onboarding design — see the open flag in §4.3.
- **CBT engine UX** (offline-first monolithic question-bundle download, relative-tick anti-cheat timer, split-screen layout for English comprehension passages, floating calculator, crash/interruption recovery banner, "Syllabus Sculptor" custom practice mode with hard 4-hour cap) — folded into §4.4.
- **Copyright/legal risk on textbooks** flagged early (New School Chemistry/Physics/Biology etc. are copyrighted; past questions are public domain) — folded into §3.4.
- **Market positioning vs. free AI** (Meta AI/WhatsApp/ChatGPT) — the "Socratic not spoon-feeding" + curriculum alignment + continuous mastery tracking argument as

the actual moat, since raw answer-generation is a commodity SabiPass cannot out-compete on price or convenience — folded into §2.

- **Marketing:** zero-budget grassroots ideas (tutorial centers near University of Ibadan, WhatsApp/Telegram JAMB candidate groups, class-rep free-account trades) — noted, not part of technical architecture, revisit when ready to launch.
-

8. Immediate next steps (as of this document's creation)

Priority order suggested, not mandatory:

1. **§4.1 — RAG/vector/memory architecture.** Highest leverage, unblocks monetization.
2. **§4.2 — Monetization,** once §4.1 gives real cost-per-user numbers.
3. **§4.3 — Explicitly resolve** whether any viral/referral growth mechanic still has a home, and where.
4. Reconcile §4.4 (CBT engine) against the new locked-account-required onboarding model.
5. §4.6 — Decide the actual guardian-consent placement/copy (NDPA-relevant, don't let this slip to "later" indefinitely).